

Algorithms

Chapter 6 Heapsort

Associate Professor: Ching-Chi Lin

林清池 副教授

chingchi.lin@gmail.com

Department of Computer Science and Engineering
National Taiwan Ocean University

Outline

- ▶ **Heaps**
- ▶ Maintaining the heap property 維護性質
- ▶ Building a heap 建立 heap
- ▶ The heapsort algorithm
- ▶ Priority queues 建立優先權佇列

The purpose of this chapter

- ▶ In this chapter, we introduce the **heapsort** algorithm.
 - ▶ with worst case running time $O(n \lg n)$ 最差情形
 - ▶ an **in-place** sorting algorithm: only a constant number of array elements are stored outside the input array at any time. 原地的(不需要其他額外的空間)
 - ▶ thus, require at most $O(1)$ additional memory 只有少數資料會放在 input array 以外的地方 (在任何時間點)
所以, 只需要 $O(1)$ 的額外空間
- ▶ We also introduce the **heap** data structure.
 - ▶ an useful data structure for heapsort 2個用途 { 用在 heap sort
 - ▶ makes an efficient priority queue 用在 priority queue

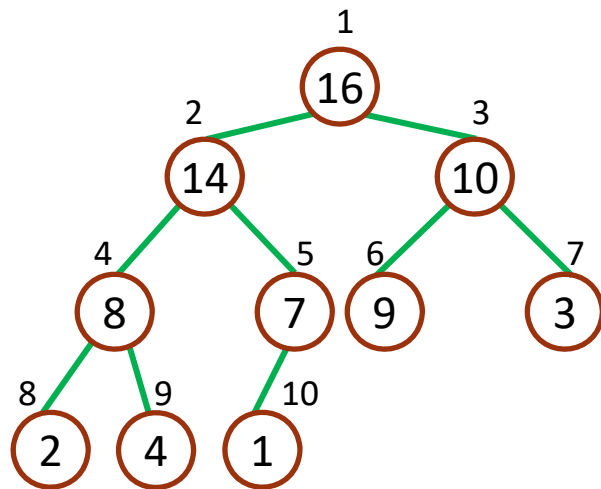
Heaps 事實上是個 array, 但我們將它看成 complete binary tree

► The (**Binary**) **heap** data structure is an **array** object that can be viewed as a nearly complete binary tree.

► A binary tree with n nodes and depth k is **complete** iff its nodes correspond to the nodes numbered from 1 to n in the full binary tree of depth k .

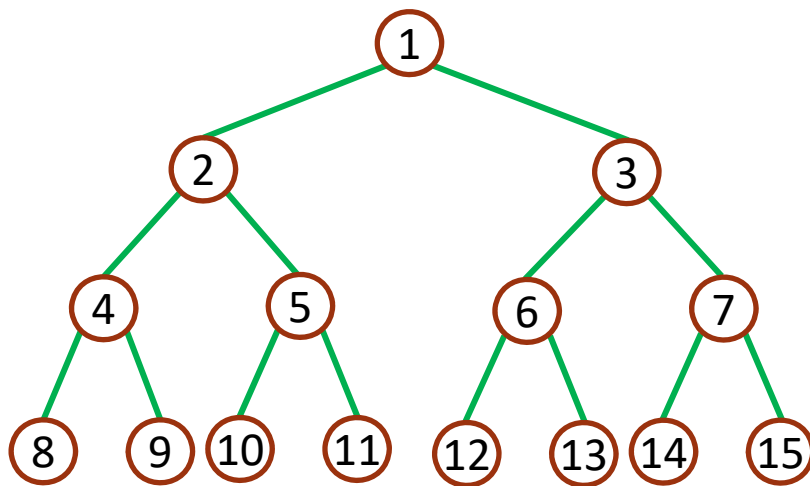
n個節點, 深度為k的二元樹為complete

⇔這n個節點與深度為k的full binary tree的節點相對應



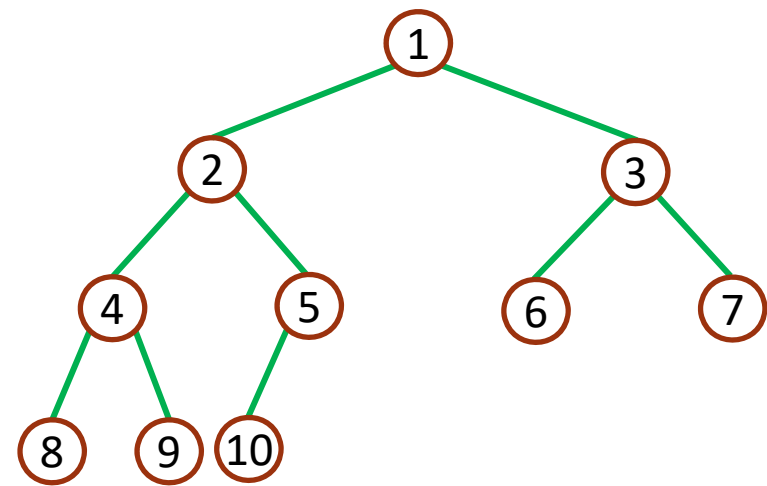
1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1

Binary tree representations



A full binary tree of height 3.

root → leaf 經過的 edge 數

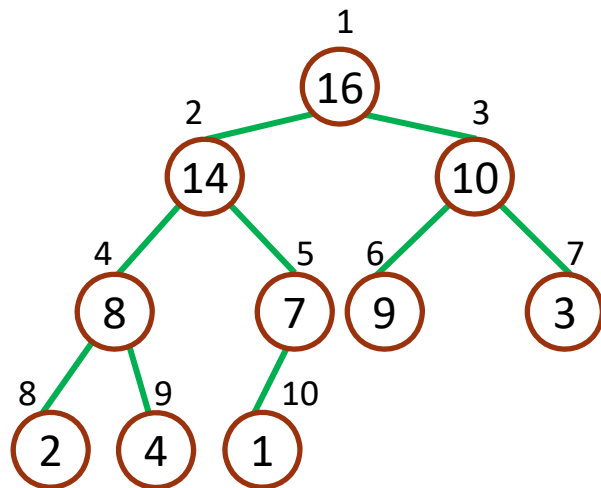


A complete binary tree with 10 nodes and height 3.

array表示heap時需要2個變數 (I) $\text{length}[A]$
(II) $\text{heap-size}[A]$

Attributes of a Heap

- ▶ An array A that presents a heap with two attributes:
 - ▶ $\text{length}[A]$: the number of elements in the array.
size (總共可以放幾個)
 - ▶ $\text{heap-size}[A]$: the number of elements in the heap stored with array A.
現在放了多少個
 - ▶ $\text{length}[A] \geq \text{heap-size}[A]$ array 的大小要比 heap-size 大

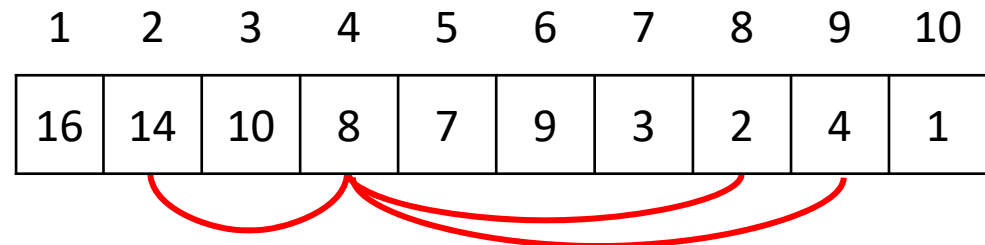
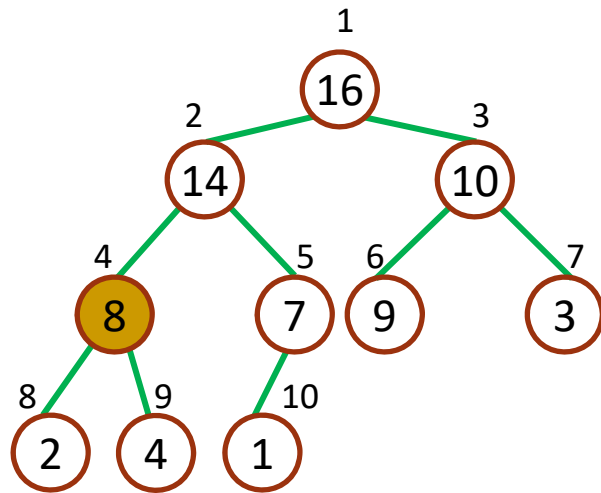


1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1

$\text{length}[A] = \text{heapsize}[A] = 10$

Basic procedures_{1/2} 對於 complete binary tree 有此性質

- ▶ If a complete binary tree with n nodes is represented sequentially, then for any node with index i , $1 \leq i \leq n$, we have
 - ▶ $A[1]$ is the **root** of the tree
 - ▶ the parent **PARENT(i)** is at $\lfloor i/2 \rfloor$ if $i \neq 1$ 父節點的 index = 子節點 / 2
 - ▶ the left child **LEFT(i)** is at $2i$ 左子節點 index = 父節點 index $\times 2$
 - ▶ the right child **RIGHT(i)** is at $2i+1$ 右子節點 index = 父節點 index $\times 2 + 1$



Basic procedures_{2/2}

$\frac{i}{2}$	0000	0010	2
i	0000	0100	4
$2i$	0000	1000	8
$2i+1$	0000	1001	9

- ▶ The **LEFT** procedure can compute $2i$ in one instruction by simply shifting the binary representation of i left one bit position.
算左子節點時, 往左移一個 bit Ex: 0100 → 1000
- ▶ Similarly, the **RIGHT** procedure can quickly compute $2i+1$ by shifting the binary representation of i left one bit position and adding in a 1 as the low-order bit.
算右子節點時, 往左移一個 bit 再加 1 Ex: 0100 → 1001
- ▶ The **PARENT** procedure can compute $\lfloor i/2 \rfloor$ by shifting i right one bit position. 算父節點時往右移 1 bit Ex: 0100 → 0010

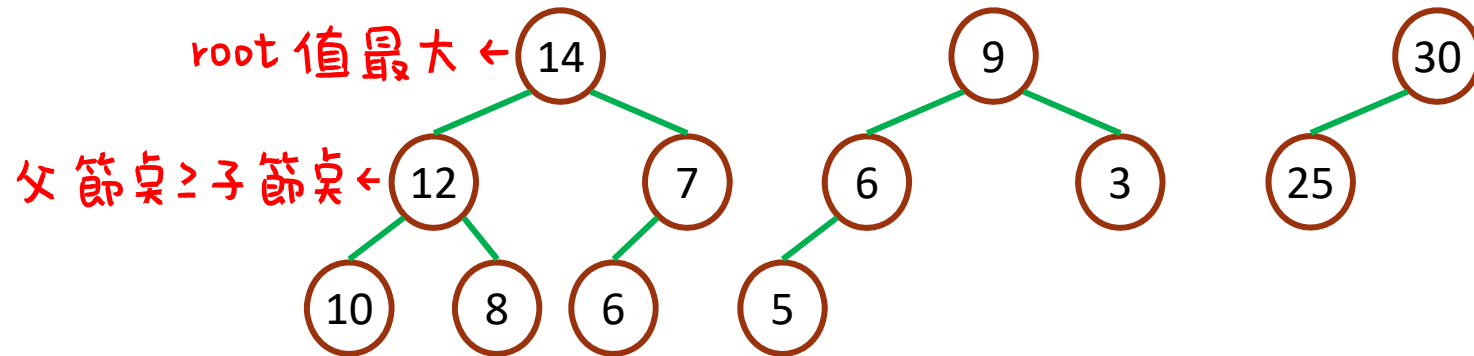
除右移
乘左移

heap 的種類 [Max heap
min heap

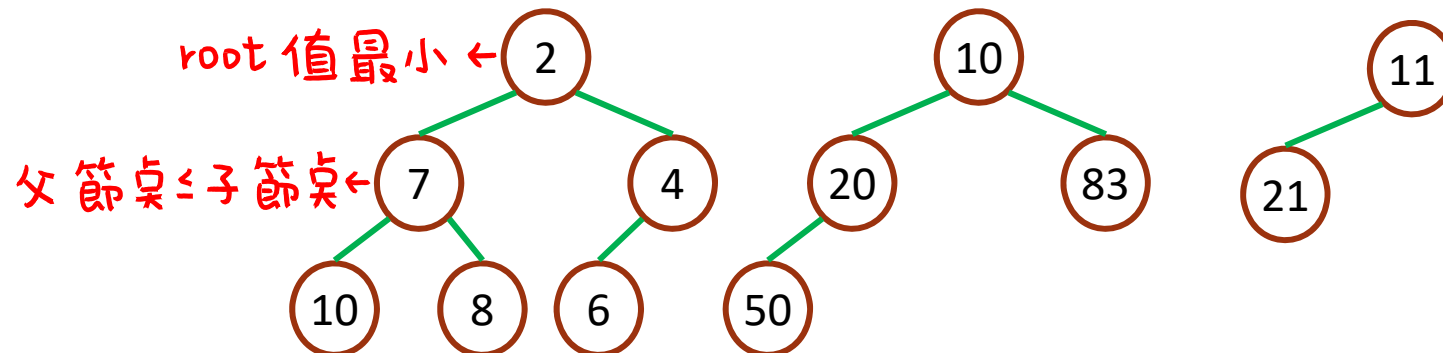
Heap properties

- ▶ There are two kind of binary heaps: max-heaps and min-heaps.
- ▶ In a **max-heap**, the **max-heap property** is that for every node i other than the root, **Max-heap property**: 父節點 $\geq$ 子節點
 $A[\text{PARENT}(i)] \geq A[i]$.
 - ▶ the largest element in a max-heap is stored at the root 在 Max-heap 中, root 最大
 - ▶ the subtree rooted at a node contains values no larger than that contained at the node itself 對於每個子樹, 子樹的 root $\geq$ 子樹中其他點
- ▶ In a **min-heap**, the **min-heap property** is that for every node i other than the root, **min-heap property**: 父節點 $\leq$ 子節點
 $A[\text{PARENT}(i)] \leq A[i]$.
 - ▶ the smallest element in a min-heap is at the root 在 min-heap 中, root 最小
 - ▶ the subtree rooted at a node contains values no smaller than that contained at the node itself 對於每一個子樹, 子樹的 root $\leq$ 子樹中其他點

Max and min heaps



Max Heaps 父節點必大於子節點



Min Heaps 父節點必小於子節點

node 的高度為到 leaf 經過的最多 edge 數

The height of a heap

heap 的高度就是 root 的高度

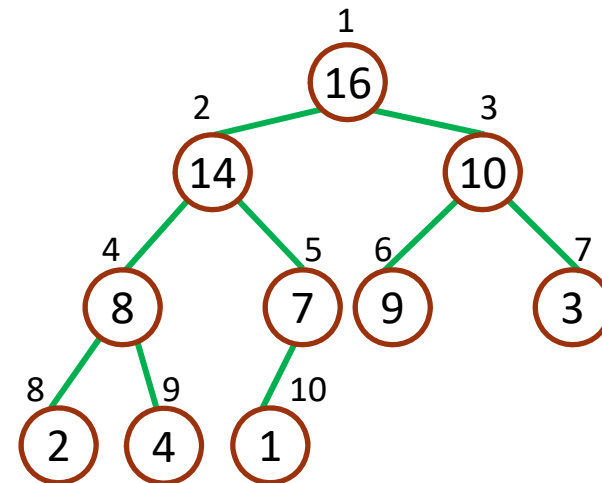
- ▶ The **height** of a node in a heap is the number of edges on the longest simple downward path from the node to a leaf, and the height of the heap to be the height of the root, that is $\Theta(\lg n)$.

n 個元素的 heap, 高度為 $\lceil \lg n \rceil = \Theta(\lg n)$ 以 2 為底

- ▶ For example:

Ex: $\lceil \lg 10 \rceil = \lceil 3.32 \rceil = 4$, heap 的高度為 4

- ▶ the height of node 2 is 2
- ▶ the height of the heap is 3



↑ 高度為 3 的 heap

The remainder of this chapter

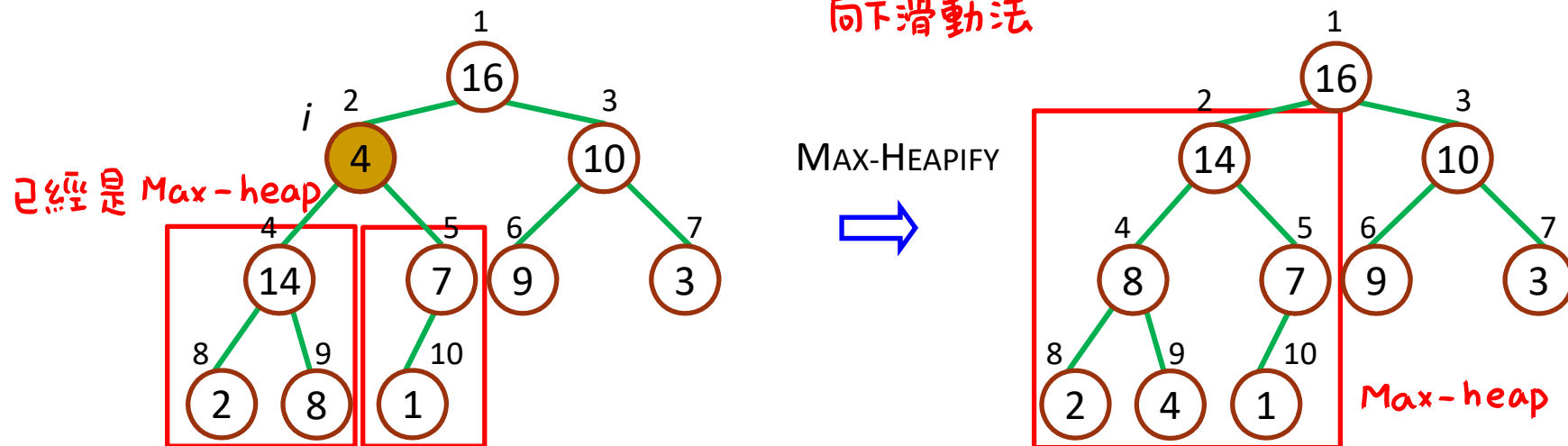
- ▶ We shall presents some basic procedures in the remainder of this chapter.
 - ▶ The **MAX-HEAPIFY** procedure, which runs in $O(\lg n)$ time, is the key to maintaining the max-heap property. 維護性質
 - ▶ The **BUILD-MAX-HEAP** procedure, which runs in $O(n)$ time, produces a max-heap from an unordered input array. 建立 heap
 - ▶ The **HEAPSORT** procedure, which runs in $O(n \lg n)$ time, sorts an array in place.
 - ▶ The **MAX-HEAP-INSERT**, **HEAP-EXTRACT-MAX**, **HEAP-INCREASE-KEY**, and **HEAP-MAXIMUM** procedures, which run in $O(\lg n)$ time, allow the heap data structure to be used as a priority queue.

Outline

- ▶ Heaps
- ▶ **Maintaining the heap property** 維護 heap 的性質
- ▶ Building a heap
- ▶ The heapsort algorithm
- ▶ Priority queues

The MAX-HEAPIFY procedure_{1/2}

- ▶ MAX-HEAPIFY is an important subroutine for manipulating max heaps.
- ▶ **Input:** an array A and an index i
- ▶ **Output:** the subtree rooted at index i becomes a max heap
以 i 為 root 的 subtree 為 max-heap
- ▶ **Assume:** the binary trees rooted at $\text{LEFT}(i)$ and $\text{RIGHT}(i)$ are max-heaps, but $A[i]$ may be smaller than its children
- ▶ **Method:** let the value at $A[i]$ “float down” in the max-heap

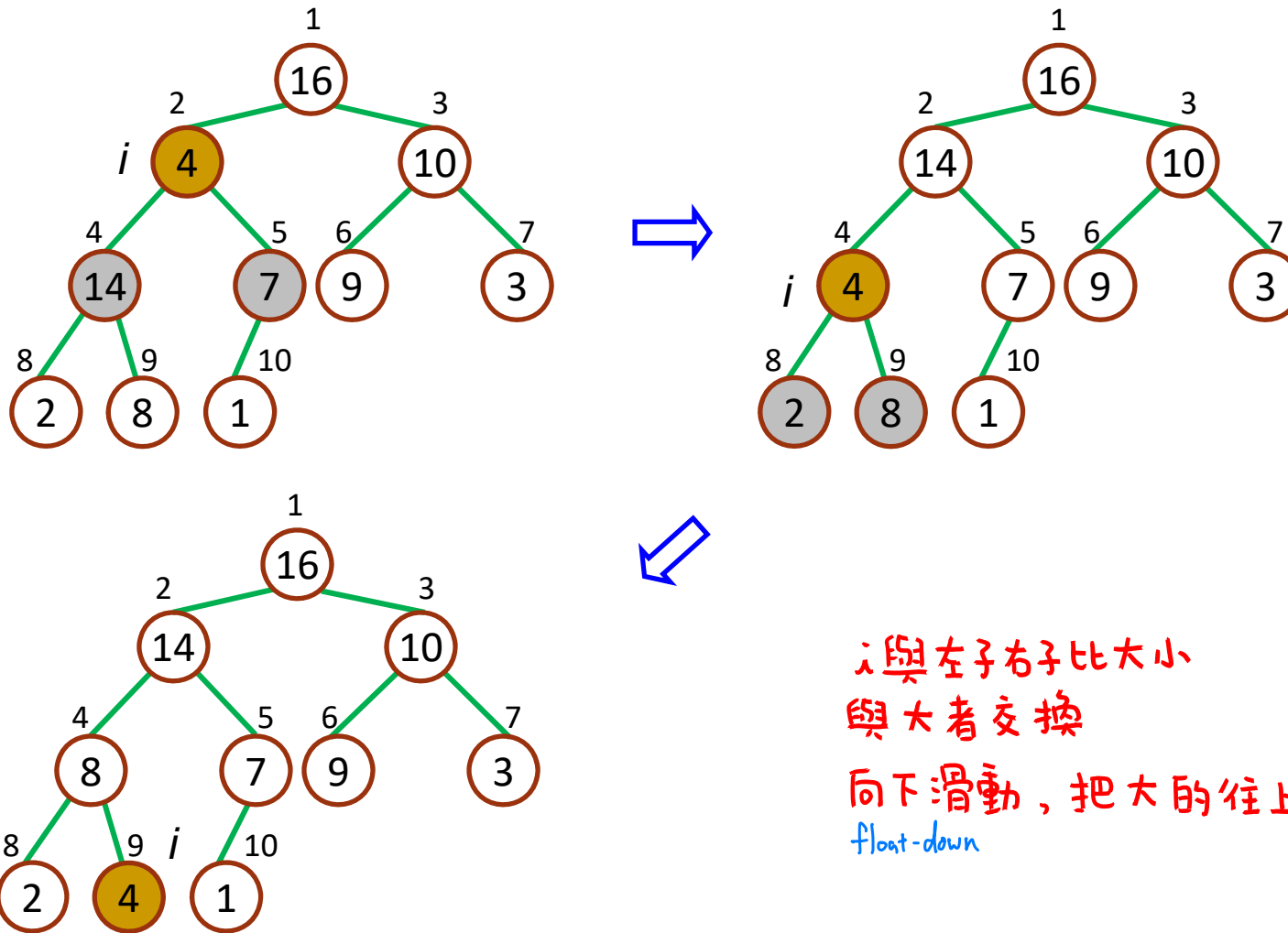


The MAX-HEAPIFY procedure_{2/2}

MAX-HEAPIFY(A, i)

1. $\ell \leftarrow \text{LEFT}(i)$ 左子 ($2i$)
2. $r \leftarrow \text{RIGHT}(i)$ 右子 ($2i+1$)
3. if $\ell \leq \text{heap-size}[A]$ and $A[\ell] > A[i]$ } $\text{largest} = \max\{\ell, i\}$
4. 有左子存在 (有值) then $\text{largest} \leftarrow \ell$ } 跟左子相比
5. else $\text{largest} \leftarrow i$
6. if $r \leq \text{heap-size}[A]$ and $A[r] > A[\text{largest}]$ } $\text{largest} = \max\{\text{largest}, r\}$
7. 有右子存在 (有值) then $\text{largest} \leftarrow r$ } 跟右子相比
8. if $\text{largest} \neq i$ 如果子節點較大
9. then exchange $A[i] \leftrightarrow A[\text{largest}]$
10. MAX-HEAPIFY ($A, \text{largest}$)

An example of MAX-HEAPIFY procedure



i 與左子右子比大小
與大者交換
向下滑動，把大的往上放
float-down

The time complexity

- ▶ It takes $\Theta(1)$ time to fix up the relationships among the elements $A[i]$, $A[\text{LEFT}(i)]$, and $A[\text{RIGHT}(i)]$.
跟子節點比大小, 做置換, 花 $\Theta(1)$ 時間
- ▶ We can characterize the running time of MAX-HEAPIFY on a node of height h as $O(h)$, that is $O(\lg n)$. 以高度來看 $O(h)$
高度為 h 的 heap 最多做 h 次置換

Outline

- ▶ Heaps
- ▶ Maintaining the heap property
- ▶ **Building a heap** 建立 heap
- ▶ The heapsort algorithm
- ▶ Priority queues

Building a Heap

- ▶ We can use the MAX-HEAPIFY procedure to convert an array $A=[1..n]$ into a max-heap in a **bottom-up** manner.
用 Max-heapify 由下往上建 (從最後一個有孩子的點往前做)
- ▶ The elements in the subarray $A[(\lfloor n/2 \rfloor + 1) \dots n]$ are all **leaves** of the tree, and so each is a 1-element heap.

$\lfloor n/2 \rfloor + 1, \lfloor n/2 \rfloor + 2 \dots n$ 皆為 leaf, 不用比

- ▶ The procedure BUILD-MAX-HEAP goes through the remaining nodes of the tree and runs MAX-HEAPIFY on each one.

$\lfloor n/2 \rfloor$ 是最後一個有子節的點, 從 $\lfloor n/2 \rfloor$ 一直往前比到 root
(孩子)

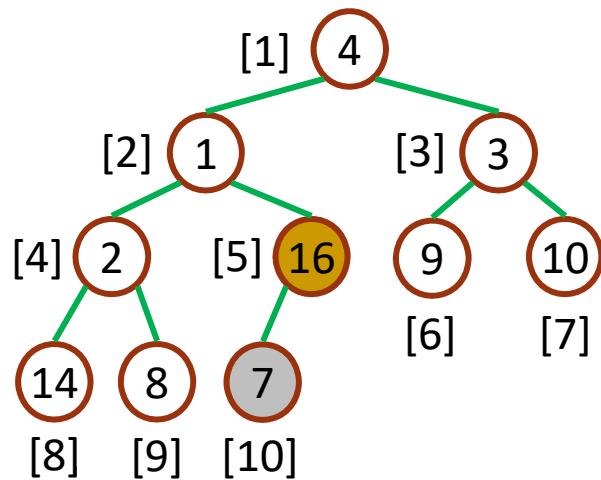
BUILD-MAX-HEAP(A)

1. $heap-size[A] \leftarrow length[A]$
2. **for** $i \leftarrow \lfloor length[A]/2 \rfloor$ **downto** 1
3. **do** MAX-HEAPIFY(A, i)

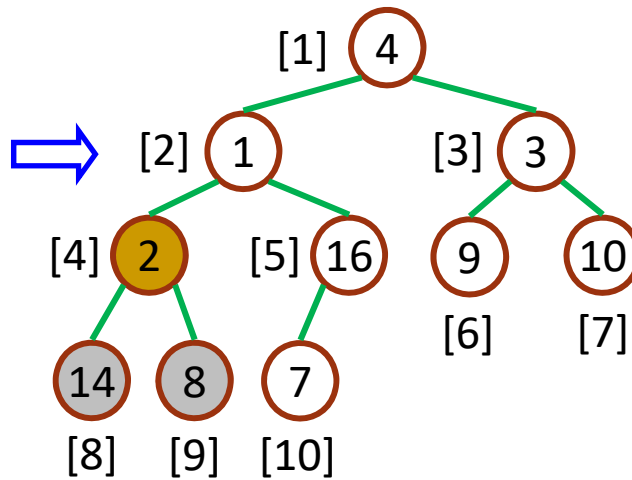
=
root

需由後往前做 MAX-HEAPIFY
如此, 在做節點 2 時, 左子樹
和右子樹都是 MAX HEAP
⇒ 滿足 MAX-HEAPIFY(A, 2) 的要求

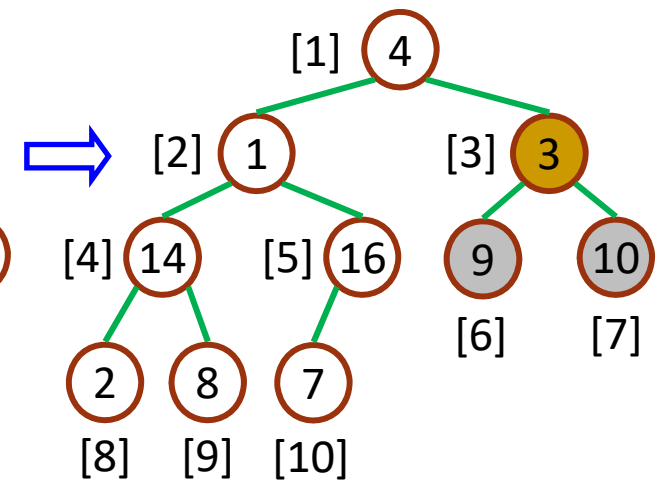
$\lfloor \frac{10}{2} \rfloor = 5$, 最後一個有子節的葉



從最後一個有子節的開始比
MAX-HEAPIFY(A, 5)



MAX-HEAPIFY(A, 4)

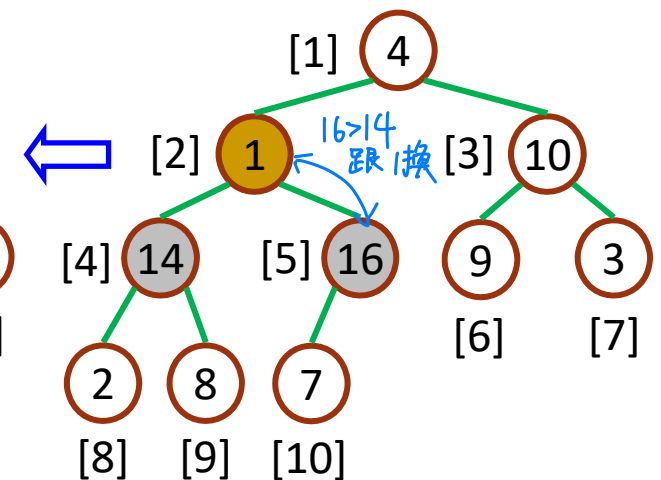
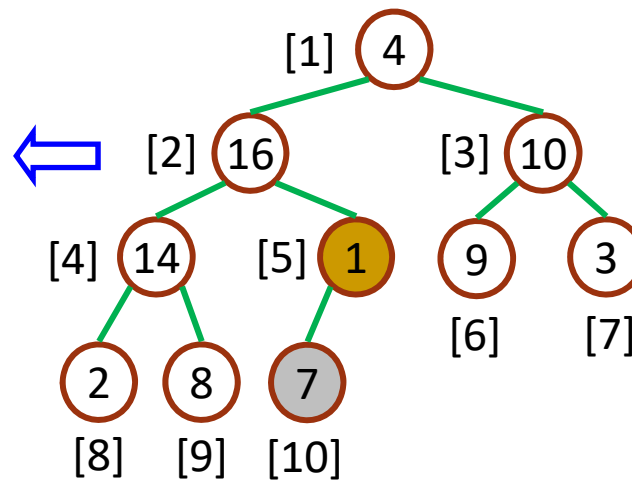
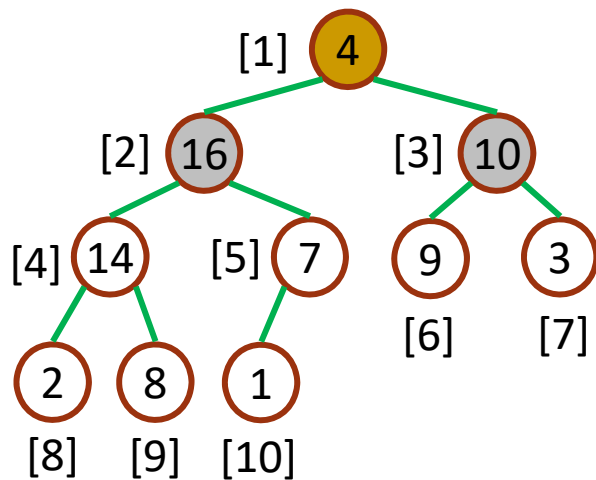


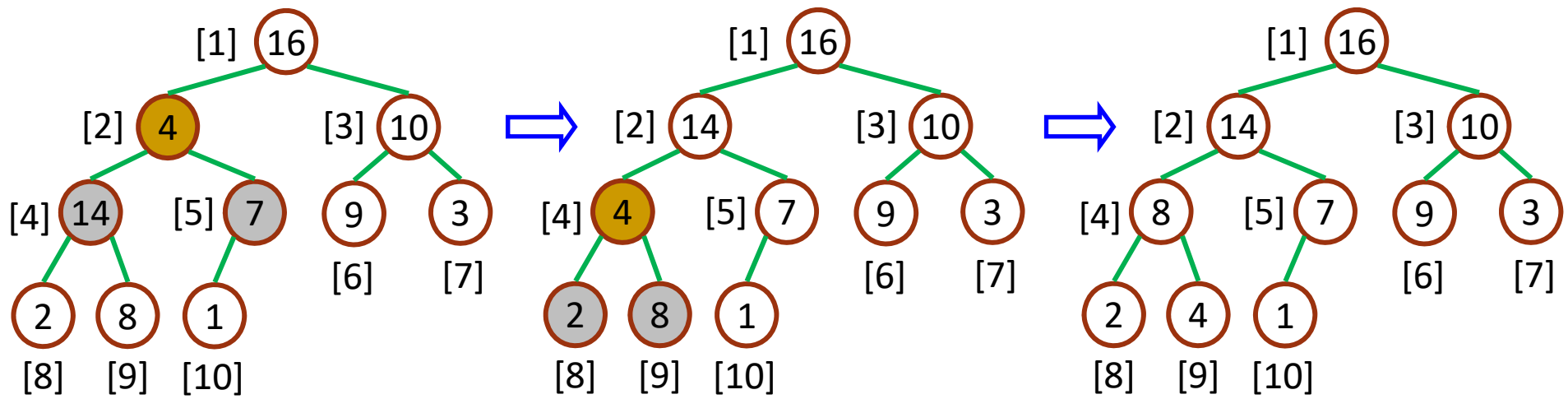
MAX-HEAPIFY(A, 3)



MAX-HEAPIFY(A, 1)

MAX-HEAPIFY(A, 2)





max-heap

Correctness_{1/2}

- ▶ To show why BUILD-MAX-HEAP work correctly, we use the following **loop invariant**: 先有一個敘述
 - ▶ At the start of each iteration of the for loop of lines 2-3, each node $i+1, i+2, \dots, n$ is the root of a max-heap.

BUILD-MAX-HEAP(A)

1. $\text{heap-size}[A] \leftarrow \text{length}[A]$
2. **for** $i \leftarrow \lfloor \text{length}[A]/2 \rfloor$ **downto** 1
3. **do** MAX-HEAPIFY(A, i)

做第 i 步時以 $i+1, i+2, \dots, n$
為 root 的 subtree 都是 Max-heap
Ex: $i=5$, 以 6, 7, 8, 9, 10 為 root
的 subtree 都是 Max-heap

- ▶ We need to show that
 - ▶ this invariant is true prior to the first loop iteration 在第一個 loop 前正確
 - ▶ each iteration of the loop maintains the invariant 在每個 loop 後仍正確
 - ▶ the invariant provides a useful property to show correctness when the loop terminates. 利用敘述來證明正確性

Correctness_{2/2}

- ▶ **Initialization:** Prior to the first iteration of the loop, $i = \lfloor n/2 \rfloor$.
 $\lfloor n/2 \rfloor + 1, \dots, n$ is a leaf and is thus the root of a trivial max-heap.

因為從第一個有子節的葉開始做, $i = \lfloor n/2 \rfloor$, 所以 $\lfloor n/2 \rfloor + 1, \lfloor n/2 \rfloor + 2 \dots n$ 都是 leaf,

- ▶ **Maintenance:** By the loop invariant, the children of node i are 自然是
both roots of max-heaps. This is precisely the Max-heap
condition required for the call MAX-HEAPIFY(A, i)
to make node i a max-heap root. Moreover, the
MAX-HEAPIFY call preserves the property that
nodes $i + 1, i + 2, \dots, n$ are all roots of max-heaps.

因為 max-heapify 的關係, 所以 $i, i+1, \dots, n$ 為 root 的 subtree 都是 max-heap

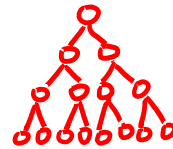
- ▶ **Termination:** At termination, $i=0$. By the loop invariant, each node
 $1, 2, \dots, n$ is the root of a max-heap.

In particular, node 1 is.

在程式結束時 $i=0$, 所以以 $1, 2, 3 \dots n$ 為 root 的 subtree 都是 max-heap
因為以 1 為 root 就是整棵樹, 故得證

Time complexity_{1/2}

Ex:



$n=15$

$\lfloor \lg 15 \rfloor = \lfloor 3.9 \rfloor = 3$

h 高度	第 h 層 node 數
0	8
1	4
2	2
3	1

$\frac{\lceil 15 \rceil}{2^{0+1}} = \lceil 7.5 \rceil = 8$

► Analysis 1: 粗略的分析

- Each call to MAX-HEAPIFY costs $O(\lg n)$, and there are $O(n)$ such calls. 每次呼叫 max-heapify 花費 $O(\lg n)$, 最多呼叫 $O(n)$ 次
- Thus, the running time is $O(n \lg n)$. This upper bound, through correct, is **not asymptotically tight**.

► Analysis 2: 花費時間 $\leq c \cdot n \lg n$, 但分析不緊密, 不夠好

- For an n -element heap, height is $\lfloor \lg n \rfloor$ and at most $\lceil n / 2^{h+1} \rceil$ nodes of any height h . n 個 node 的高度為 $\lfloor \lg n \rfloor$, 第 h 層 node 數最多為 $\lceil n / 2^{h+1} \rceil$ 個 nodes
- The time required by MAX-HEAPIFY when called on a node of height h is $O(h)$.

- The total cost is $\sum_{h=0}^{\lfloor \lg n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil O(h) = O\left(n \sum_{h=0}^{\lfloor \lg n \rfloor} \frac{h}{2^h}\right)$.

將每層加起來

Time complexity_{2/2}

- The last summation yields

$$\sum_{k=0}^{\infty} k x^k = \frac{x}{(1-x)^2}, \text{ for } |x| < 1$$

$$x + 2x^2 + 3x^3 + \dots$$

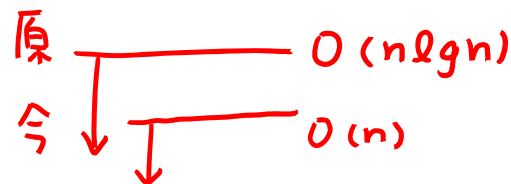
$$\sum_{h=0}^{\infty} \frac{h}{2^h} = \frac{1/2}{(1-1/2)^2} = 2$$

公式 [法1]

- Thus, the running time of BUILD-MAX-HEAP can be bounded as

$$\sum_{h=0}^{\lfloor \lg n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil O(h) = O\left(n \sum_{h=0}^{\infty} \frac{h}{2^h}\right) = O(n)$$

- We can build a max-heap from an unordered array in **linear time**.



⇒ 上界推算出往下了, 更加緊密

[法2]

$$S = \frac{1}{2} + \frac{2}{4} + \frac{3}{8} + \dots$$

$$-\frac{1}{2}S = -\frac{1}{4} + \frac{2}{8} + \dots$$

$$\frac{1}{2}S = \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots$$

$$= \frac{\frac{1}{2}}{1 - \frac{1}{2}} = 1 \Rightarrow S = 2$$

Outline

- ▶ Heaps
- ▶ Maintaining the heap property
- ▶ Building a heap
- ▶ **The heapsort algorithm**
- ▶ Priority queues

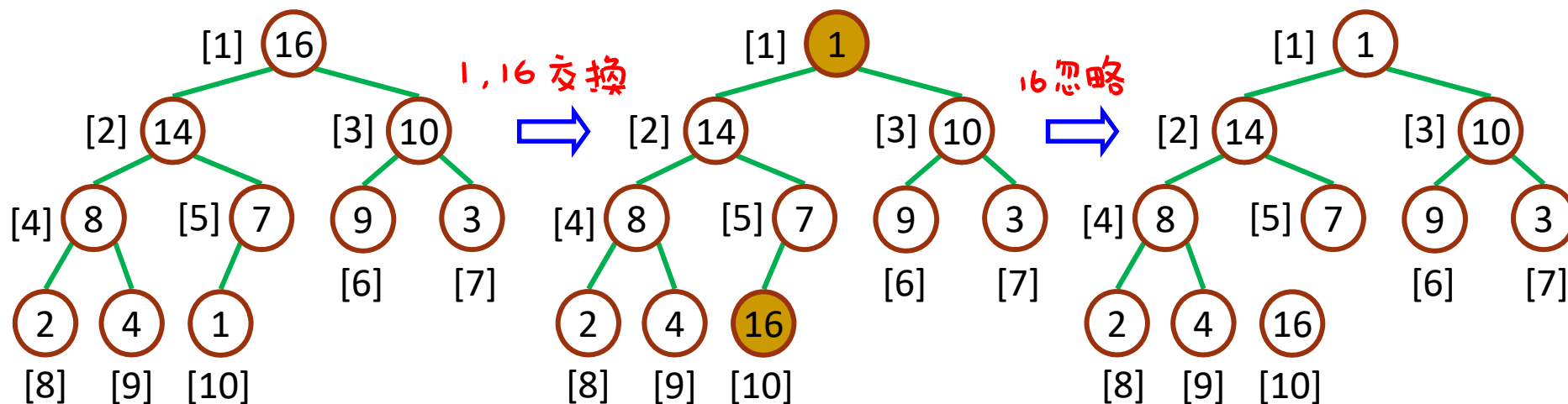
3 step : (I) 交換 exchange
(II) 忽略 discard
(III) 重新調整 readjust

The heapsort algorithm

- ▶ Since the maximum element of the array is stored at the root, $A[1]$ we can **exchange** it with $A[n]$.
- ▶ If we now “**discard**” $A[n]$, we observe that $A[1...(n-1)]$ can easily be made into a max-heap.
- ▶ The children of the root $A[1]$ remain max-heaps, but the new root $A[1]$ element may violate the max-heap property, so we need to **readjust** the max-heap. That is to call MAX-HEAPIFY($A, 1$).

HEAPSORT(A)

1. BUILD-MAX-HEAP(A)
 2. **for** $i \leftarrow \text{length}[A]$ **downto** 2 從 n 往前做到 2
 3. **do** exchange $A[1] \leftrightarrow A[i]$
 4. $\text{heap-size}[A] \leftarrow \text{heap-size}[A] - 1$
 5. MAX-HEAPIFY($A, 1$)
- } 交換
忽略 將 $\text{heap-size} - 1$
重新調整



Initial heap

Exchange

Heap size = 10
Sorted=[16]

Discard

Heap size = 9
Sorted=[16]

Discard

Heap size = 8
Sorted=[14,16]

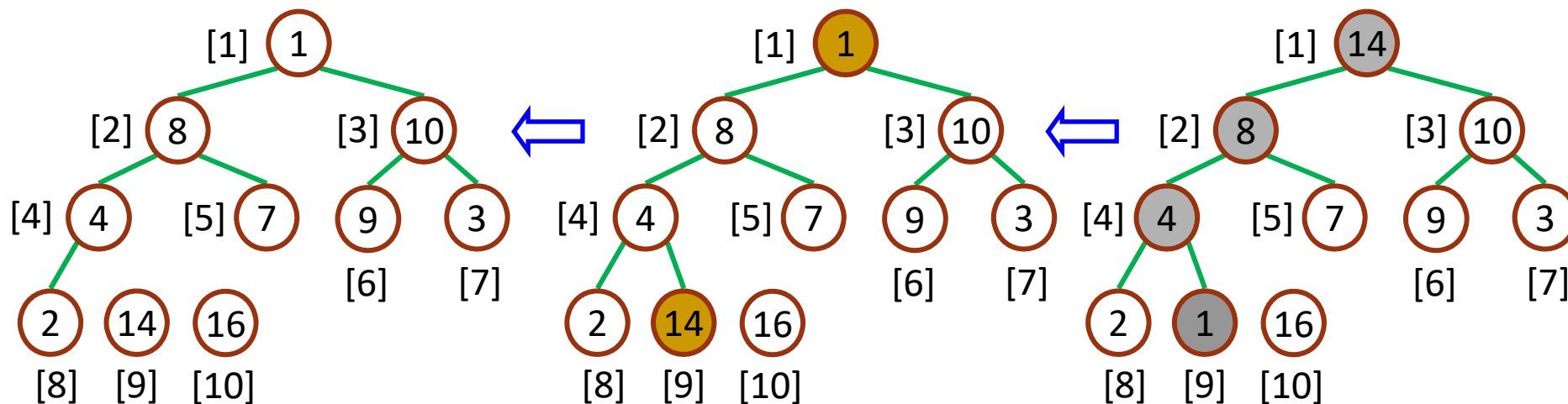
Exchange

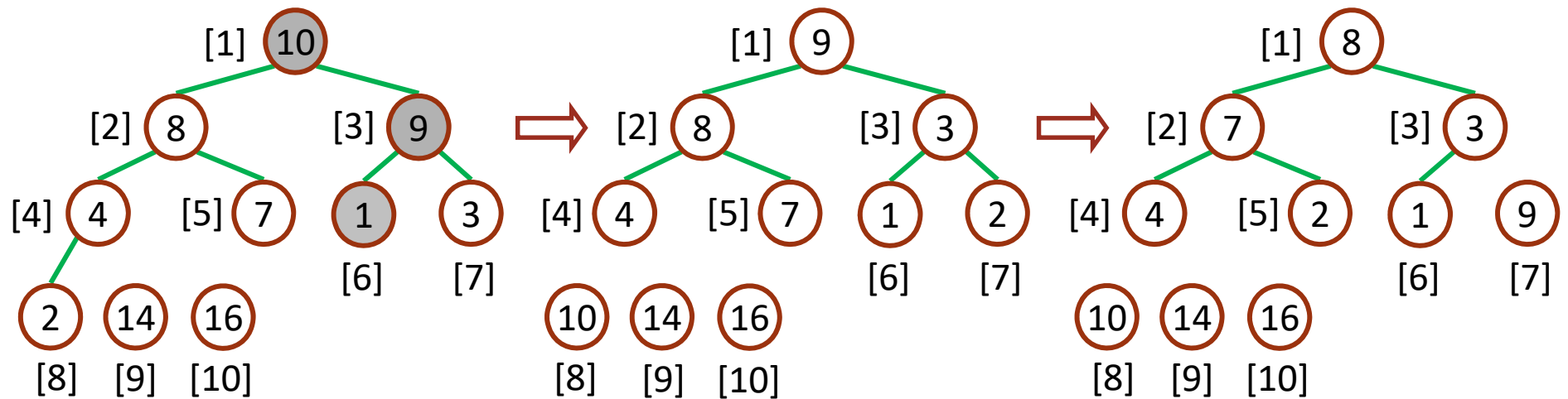
Heap size = 9
Sorted=[14,16]

Readjust

Heap size = 9
Sorted=[16]

重新調整





Readjust
 Heap size = 8
 Sorted=[14,16]

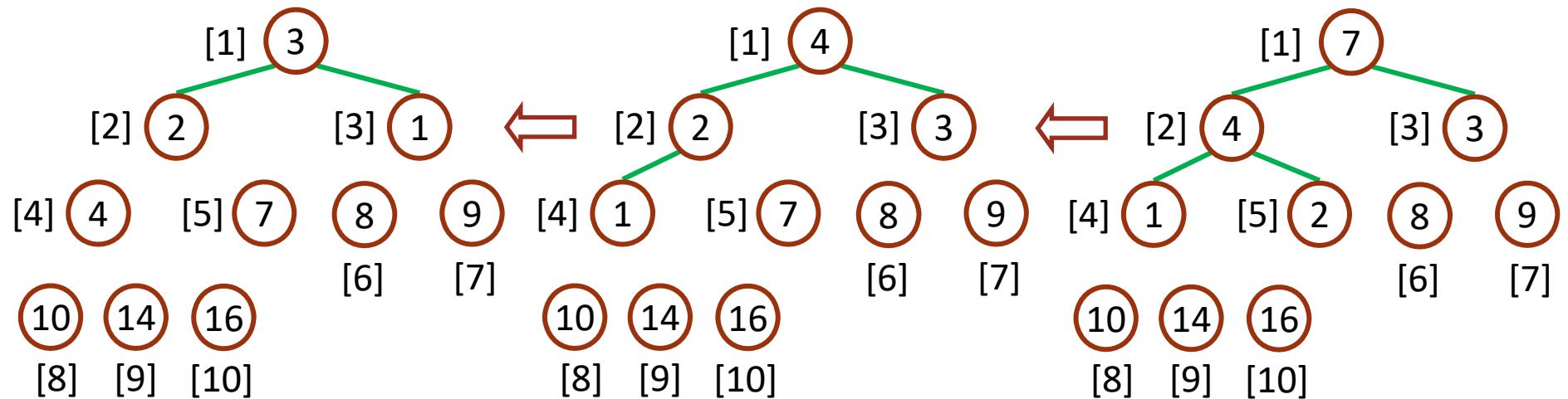
Heap size = 7
 Sorted=[10,14,16]

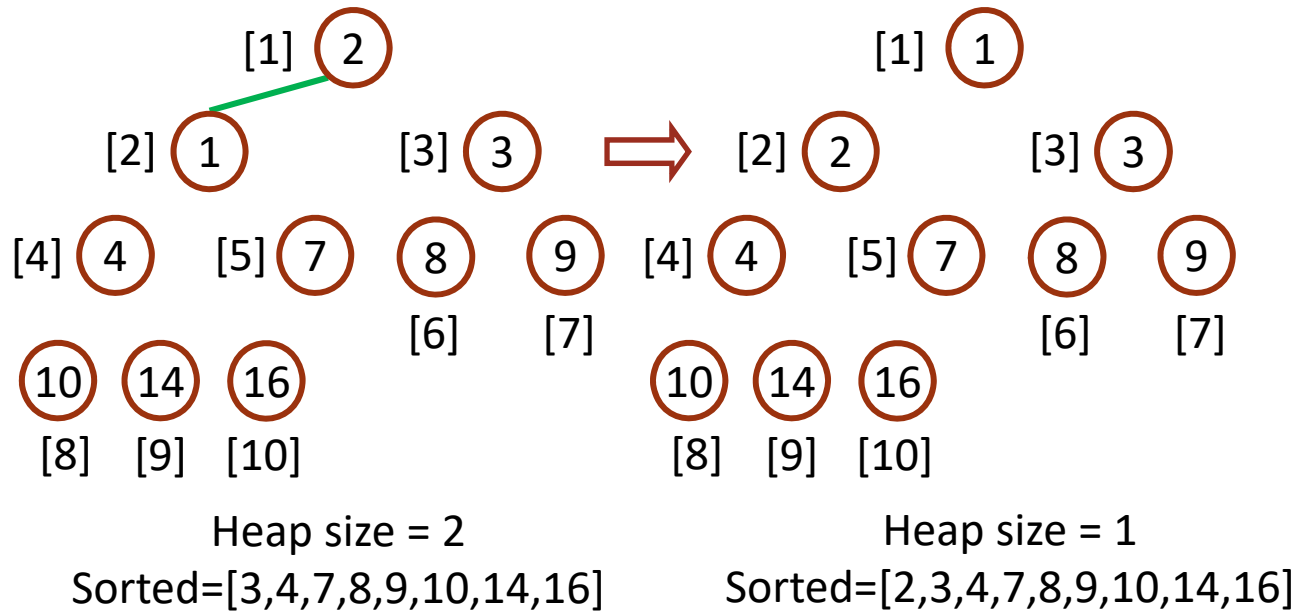
Heap size = 6
 Sorted=[9,10,14,16]

Heap size = 3
 Sorted=[4,7,8,9,10,14,16]

Heap size = 4
 Sorted=[7,8,9,10,14,16]

Heap size = 5
 Sorted=[8,9,10,14,16]





Time complexity $O(n) + (n-1) \cdot O(\lg n) = O(n \lg n)$

- ▶ The HEAPSORT procedure takes $O(n \lg n)$ time
 - ▶ the call to BUILD-MAX-HEAP takes $O(n)$ time 建立 heap 的時間
 - ▶ each of the $n-1$ calls to MAX-HEAPIFY takes $O(\lg n)$ time
呼叫 max-heapify $n-1$ 次, 每次 $O(\lg n)$

Outline

- ▶ Heaps
- ▶ Maintaining the heap property
- ▶ Building a heap
- ▶ The heapsort algorithm
- ▶ **Priority queues** 優先佇列

Heap implementation of priority queues

- ▶ Heaps efficiently implement priority queues.
- ▶ There are two kinds of priority queues: max-priority queues and min-priority queues.
- ▶ We will focus here on how to implement max-priority queues, which are in turn based on max-heaps.
- ▶ A **priority queue** is a data structure for maintaining a set S of elements, each with an associated value called a **key**.

priority queue 是一個資料結構，用來維護一群元素，
每一個元素都有一個鍵值 (key)

priority queue 有 2 種 { Max priority queue
min priority queue

Priority queues

- ▶ A **max-priority queue** supports the following operations.
 - ▶ $\text{INSERT}(S, x)$: inserts the element x into the set S .
加入元素 x 到 set S 中
 - ▶ $\text{MAXIMUM}(S)$: returns the element of S with the largest key.
告訴我誰是最大者
 - ▶ $\text{EXTRACT-MAX}(S)$: removes and returns the element of S with the largest key. 告訴我誰是最大者, 並且將他除外
 - ▶ $\text{INCREASE-KEY}(S, x, k)$: increases value of element x 's key to the new value k . Assume $k \geq x$'s current key value.
將 x 的 key 值增加為 k [將鍵值加大]

Finding the maximum element

- ▶ $\text{MAXIMUM}(S)$: returns the element of S with the largest key.
- ▶ Getting the maximum element is easy: it's the root.

$\text{HEAP-MAXIMUM}(A)$

1. return $A[1]$ 是 max-heap, 最大者為 root

- ▶ The running time of HEAP-MAXIMUM is $\Theta(1)$.

Extracting max element

- ▶ **EXTRACT-MAX(S)**: removes and returns the element of S with the largest key.

HEAP-EXTRACT-MAX(A)

$O(1)$ $O(\lg n)$ $O(1)$

1. **if** $\text{heap-size}[A] < 1$] 檢查 heap 是否為空
2. **then error** "heap underflow"
3. $\text{max} \leftarrow A[1]$ 將最大值存起來
4. $A[1] \leftarrow A[\text{heap-size}[A]]$ 將最後一個放到 root
5. $\text{heap-size}[A] \leftarrow \text{heap-size}[A] - 1$ 將 heap-size 減 1
6. MAX-HEAPIFY($A, 1$) 做調整
7. **return** max

- ▶ **Analysis**: constant time assignments + time for MAX-HEAPIFY.

- ▶ The running time of HEAP-EXTRACT-MAX is $O(\lg n)$.

$$O(1) + O(\lg n) = O(\lg n)$$

(為某个節点)

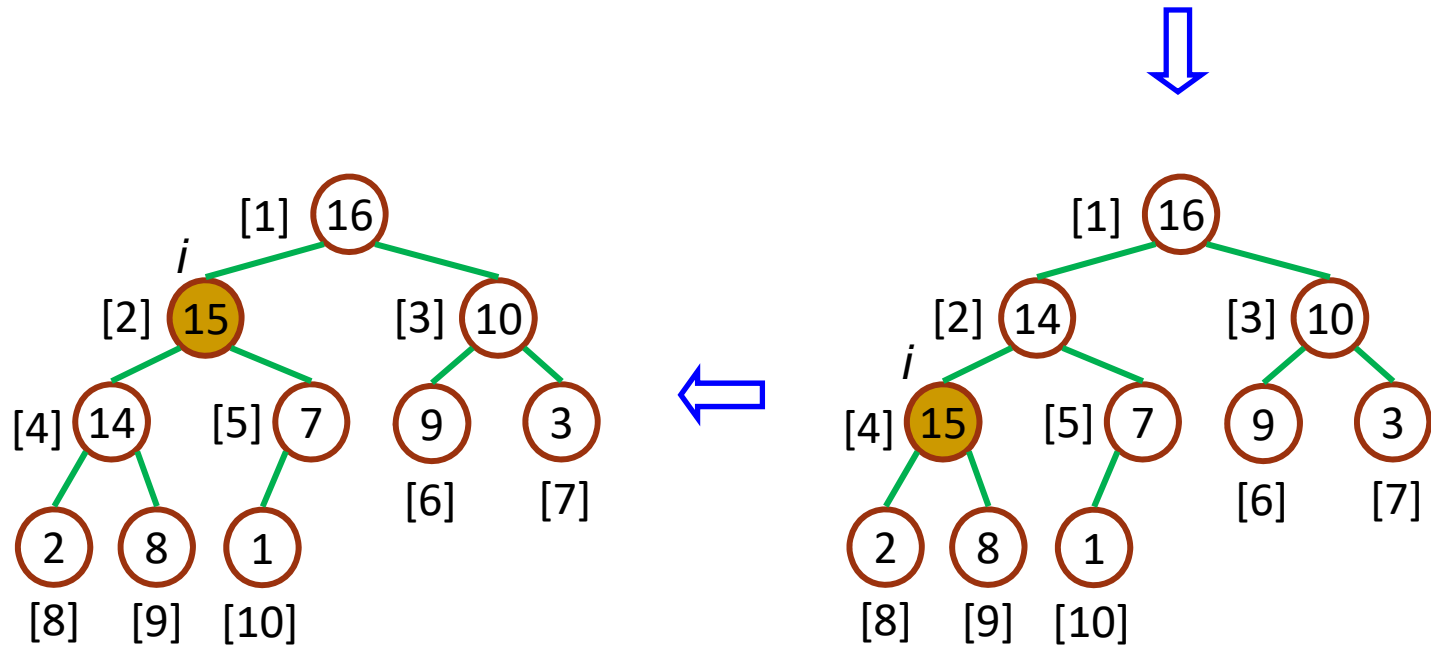
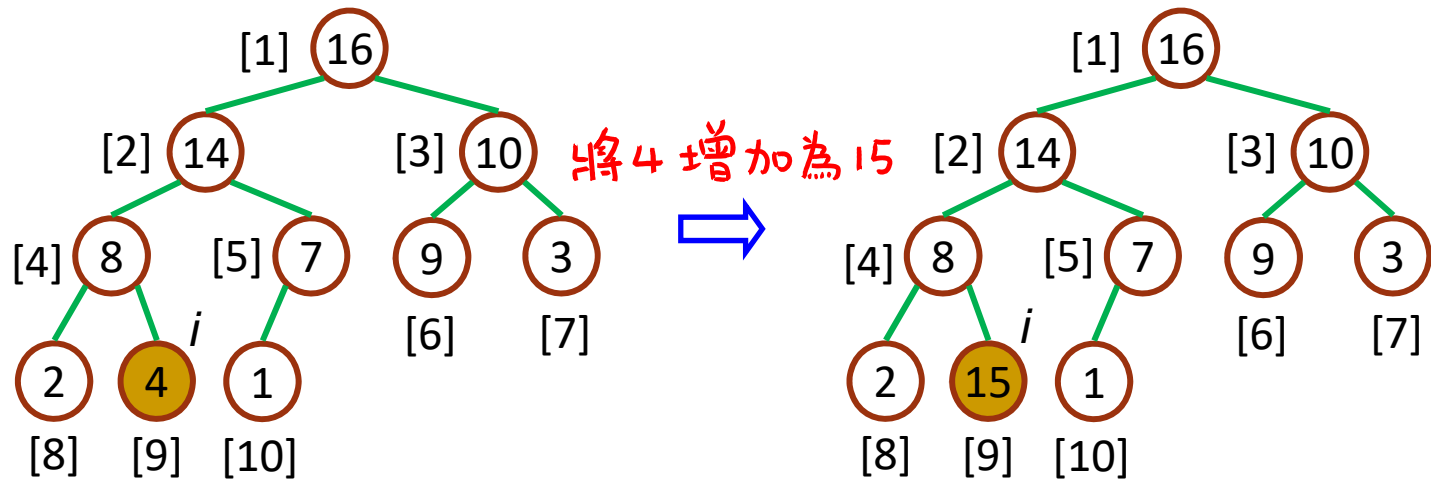
Increasing key value 增加鍵值

- ▶ INCREASE-KEY(S, x, k): increases value of element x 's key to k .
Assume $k \geq x$'s current key value.

HEAP-INCREASE-KEY (A, i, key)

1. **if** $key < A[i]$ 檢查 key 值真的有增加
2. **then error** “new key is smaller than current key”
3. $A[i] \leftarrow key$
4. **While** $i > 1$ and $A[PARENT(i)] < A[i]$ 如果不是 root, 而且 key 值比父節點大,
5. **do** exchange $A[i] \leftrightarrow A[PARENT(i)]$ 與父節點做交換
6. $i \leftarrow PARENT(i)$

- ▶ **Analysis:** the path traced from the node updated to the root has length $O(\lg n)$. 最多是到 root, 所以最多為 $O(\lg n)$ 次 [tree 的高度]
- ▶ The running time is $O(\lg n)$.



Inserting into the heap 插入一个新元素

- ▶ $\text{INSERT}(S, x)$: inserts the element x into the set S . 增加一個 element

$\text{MAX-HEAP-INSERT}(A, \text{key})$

- $O(1)$ [
1. $\text{heap-size}[A] \leftarrow \text{heap-size}[A] + 1$ 將 heap-size 加 1
 2. $A[\text{heap-size}[A]] \leftarrow -\infty$ 將最後一個放 $-\infty$
- $O(\lg n)$ 3. $\text{HEAP-INCREASE-KEY}(A, \text{heap-size}[A], \text{key})$
將最後一個的 key 值從 $-\infty$ 增加為 key 值

- ▶ **Analysis**: constant time assignments + time for HEAP-INCREASE-KEY .
- ▶ The running time is $O(\lg n)$.